PRACTICAL - 12

Aim: Word frequency (WordCounter.java + Driver.java)

Code Screenshot:

WordCounter.java

```java
import java.util.*;

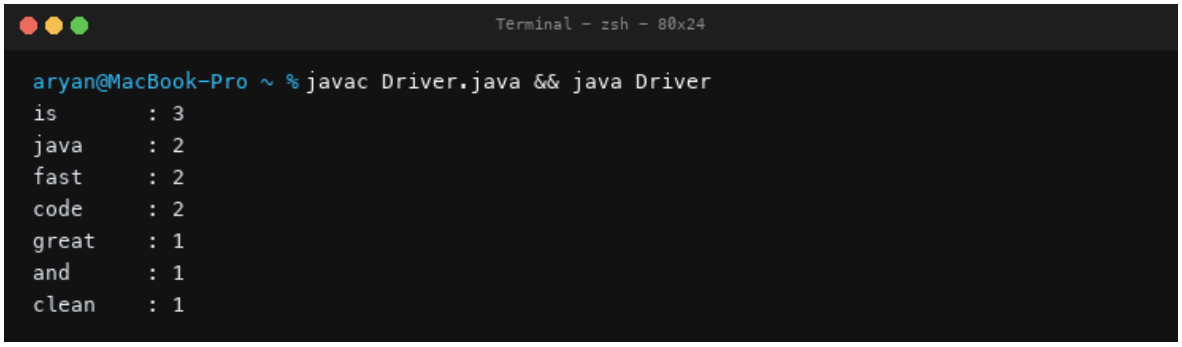
public class WordCounter {
    public static Map<String, Integer> count(String text) {
        Map<String, Integer> map = new HashMap<>();
        for (String w : text.toLowerCase().split("\\W+")) {
            if (!w.isEmpty()) map.put(w, map.getOrDefault(w, 0) + 1);
        }
        return map;
    }
}
```

Driver.java

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        String text = "Java is great and Java is fast. Fast code is clean code!";
        Map<String, Integer> freq = WordCounter.count(text);
        List<Map.Entry<String, Integer>> list = new ArrayList<>(freq.entrySet());
        list.sort(Map.Entry.<String, Integer>comparingByValue().reversed());
        for (Map.Entry<String, Integer> e : list) {
            System.out.printf("%-8s : %d\n", e.getKey(), e.getValue());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered key-value associative data storage using HashMap.
- Used map.getOrDefault() for concise frequency counter accumulation.
- Sorted map entry sets dynamically using Comparator chaining.
- Tokenized and sanitized unstructured text with regular expressions.
- Understood hash table average O(1) retrieval performance.

Aim: Unique visitors (VisitorTracker.java + Driver.java)

Code Screenshot:

VisitorTracker.java

```java
import java.util.*;

public class VisitorTracker {
    private Set<String> uniqueSet = new HashSet<>();
    private Set<String> orderedSet = new LinkedHashSet<>();

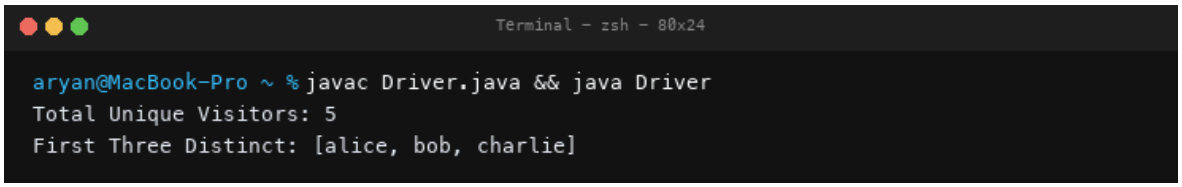
    public void recordVisit(String name) {
        uniqueSet.add(name);
        orderedSet.add(name);
    }

    public int getUniqueCount() { return uniqueSet.size(); }
    public List<String> getFirstThree() {
        return orderedSet.stream().limit(3).toList();
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        VisitorTracker tracker = new VisitorTracker();
        String[] visits = { "alice", "bob", "alice", "charlie", "bob", "david", "emma" };
        for (String v : visits) tracker.recordVisit(v);
        System.out.println("Total Unique Visitors: " + tracker.getUniqueCount());
        System.out.println("First Three Distinct: " + tracker.getFirstThree());
    }
}
```

Output Screenshot:



Learning Outcome:

- Compared unordered HashSet against insertion-ordered LinkedHashSet.
- Implemented automatic data deduplication using Set semantics.
- Preserved arrival order while filtering duplicate telemetry events.
- Streamed collection subsets cleanly with stream().limit().
- Evaluated memory and time trade-offs across distinct Java collection types.